Payment Operations Intelligence: Project Guide

Oct 16, 2026 · @nexverra

1. What the project is about

An analyst types "Why did payment failures go up last week?" and, in about half a minute on a laptop, gets a checked report: what changed, which payment provider and failure reason explain it, when it started and ended, which deployments and provider notices were nearby, what it cost, and what to do next. Everything runs on one computer, with no cloud services and no paid APIs.

The problem. Online payments go from a merchant to a payment provider, which answers approved or failed. A few percent always fail. When that rate jumps, an analyst must combine four kinds of evidence by hand: the transactions, the company's own software releases (deployments), the providers' status notices, and history to know what normal looks like. That takes hours, and the common shortcut, blaming the last release, is often wrong.

What the system does. Six agents split the work. Code and SQL compute every number. A small local LLM (`qwen3:4b` through Ollama) does only two things: turns the question into a plan, and writes the report from the computed findings. A Verification agent checks every sentence the LLM writes; if too many fail, the user gets a clearly labelled template report instead.

How we know it works. The data is synthetic (492,996 payments over 90 days, four providers) with a hidden incident and an answer key the app never reads:

Question	Hidden truth	What the system reported
Which provider?	KESTREL	KESTREL, 98% of the extra failures
Which failure reason?	TIMEOUT	TIMEOUT, 30 → 480
When?	23 Sep 14:00 → 25 Sep 02:00	23 Sep 14:00 → 25 Sep 01:00
Our deployment 2 h before?	A red herring	"Correlation only", ranked unlikely

Question	Hidden truth	What the system reported
How big?	—	KESTREL failure rate 2.73% → 23.07% in the window

Other measured results: incident detection had no false alarms across 8 random datasets (14/14 alarms real, 14/16 incidents found). With the real local model, 93.8% of LLM sentences passed verification, and both prompt-injection attempts were handled safely. Your laptop passed 435 automated tests (441 once python-pptx is installed).

2. Why agents and an LLM instead of a trained ML model

The main reason is not computing power. An LLM actually needs more computing than classic ML: `qwen3:4b` needs a GPU and several GB of memory, while a statistical model runs in milliseconds on any CPU. The real reasons are about data, explanation and flexibility, and the project still uses lightweight statistics where they fit.

What a classic ML approach would need, and why it doesn't fit here:

Need	Trained ML model (for example a root-cause classifier)	This project
Training data	Hundreds of past incidents, each labelled with its true cause. Most payment teams don't have this; incidents are rare and each one differs	No labels needed: statistics plus rules, written once
Explanation	A score or a class label; hard to say why	Every finding cites the queries and transactions behind it; hypotheses list evidence for and against
New questions	Retrain for each new kind of question	The LLM turns a plain-English question into a plan; code validates it
Auditability in finance	Model behaviour must be validated and monitored for drift	Numbers come from SQL and tested Python, so they can be checked by hand

Need	Trained ML model (for example a root-cause classifier)	This project
Compute	Low to run, but training and retraining needed	Analytics: under 0.5 s on a CPU. LLM: about 17–57 s per question on a laptop GPU

Where statistics is still used (deterministic, no training):

- a two-proportion z-test to decide whether a rise is real or noise
- a detection baseline that compares each provider's hour with that provider's failure rate over the previous 7 days, and flags hours far above it ($z \geq 4$ and at least 5 extra failures)
- a small linear forecast (9 parameters, numpy least squares) for next week's volume, which beat the naive baseline (51 vs 118 payments/day error)

Why several agents instead of one script? Each agent has one job, a fixed list of tools it may call, its own evidence and its own checkpoint. If one fails, the others still finish and the report says what is missing. It also mirrors how a real team splits an investigation. A single script would be simpler and is honestly enough for a fixed weekly report; `docs/agent_design.md` says where.

Why the LLM is kept on a short leash. A 4-billion-parameter model makes arithmetic and copying mistakes. So it never computes a number, it never decides access, and every sentence it writes must cite a finding whose numbers match, or the sentence is removed.

Where ML would add value later: with a year of real labelled incidents, a supervised model could rank likely causes, and learned seasonality could improve the forecast. Both would plug in as new tools behind the same agents.

3. Tools used in each phase, and why

Every tool is free, runs locally and needs no account or API key. The one exception is the optional GitHub Actions CI, which needs a GitHub account. Licences and network behaviour for each are in `docs/dependency_inventory.md`.

Phase	What was built	Tools	Why these
0	Plan, prerequisites	Windows 11, Python 3.11, Docker Desktop, Ollama, Git	Python is the standard for data and AI work. Docker runs PostgreSQL without installing it into Windows. Ollama runs LLMs locally on your GPU
1	Settings, local-only network rule, health check	pydantic-settings, FastAPI, uvicorn, pytest	Settings are validated at start-up, so a non-local address is refused before anything runs. pytest blocks all internet calls during tests
2	Synthetic dataset and database	numpy, pandas, PostgreSQL 16, psycopg	A seeded generator gives the same 492,996 rows every time. PostgreSQL gives real logins, read-only sessions and query time limits; `COPY` loads 500k rows in seconds
3	Deterministic analytics	SQLAlchemy, numpy	Parameterised SQL (no injection). Statistics (z-test, detection baseline, linear forecast) need no training and are tested against the hidden answer key
4	API and dashboard	FastAPI, Streamlit, Plotly	FastAPI validates every request. Streamlit builds a dashboard in pure Python; usage statistics are switched off
5a	Six agents, workflow	LangGraph	Shared state, parallel steps (three agents at once), a hard step limit; no hosted service needed

Phase	What was built	Tools	Why these
5b	Local LLM and verification	Ollama, `qwen3:4b`, httpx2	The model runs on your laptop's GPU; JSON-schema output makes replies machine-checkable; the Verification agent checks every sentence
6	History, checkpoints, resume, limits	PostgreSQL (schema `ops`), pydantic	Each step is saved as validated JSON, so an interrupted run resumes; avoids unpickling objects, a known risk class
7	Sign-in, roles, audit log, LLM evaluation	Python standard library (scrypt, secrets, hashlib), PostgreSQL triggers	No external identity provider allowed; scrypt at OWASP's parameters; the audit log is append-only and hash-chained
8	Packaging and CI	Docker, Docker Compose, GitHub Actions, pip-audit	One command starts everything; non-root containers on 127.0.0.1 only; CI tests every push and checks dependencies for known vulnerabilities
9	Docs, diagram, presentation, real-data preparation	python-pptx, pandas	An editable PowerPoint generated from a script, so numbers and speaker notes can't drift; a script to profile, clean and convert your own data

Two licence notes worth knowing: psycopg is LGPL-3.0, which allows use as an unmodified library. Docker Desktop is not open source: it is free for personal use, education and small businesses (under 250 employees and under $10M revenue); Podman is a free alternative.

4. Every folder and file

The application code lives in `src/payops/`, one folder per layer; everything else is configuration, scripts, tests and docs. The brief map comes first, then each file.

4.1 The brief map

Folder	In one line
`src/payops/`	The application: settings, network rule, logging, health
`src/payops/data/`	Generates the synthetic dataset with a hidden incident
`src/payops/db/`	Database tables, loading data, creating logins
`src/payops/analytics/`	Deterministic statistics: rates, detection, events, impact, forecast
`src/payops/agents/`	The six agents, their tools, the LangGraph workflow, the CLI
`src/payops/llm/`	The local Ollama client and the prompts
`src/payops/history/`	Saving investigations and step checkpoints
`src/payops/security/`	Passwords, sessions, roles, audit log, account commands
`src/payops/api/`	The FastAPI web service
`src/payops/dashboard/`	The Streamlit dashboard
`src/payops/evaluation/`	Measures the real LLM on 10 questions with known answers
`tests/`	448 automated tests (unit tests plus `integration/` against the real database and LLM)
`scripts/`	Build the deck, prepare real data, helpers for Docker and CI
`docs/`	Everything written down: design, security, evaluation, runbook

Folder	In one line
`sql/`	Hand-written SQL an analyst can run to find the incident without the app
`presentation/`	The generated PowerPoint
`data/`	Generated or prepared data (never committed)
`.github/workflows/`	The CI pipeline
`evaluation/results/`	Output of LLM evaluation runs

4.2 Root files

File	What it does
`README.md`	Setup and run instructions, start to finish
`pyproject.toml`	Project name, Python version, package location (`src/`), pytest settings
`requirements.in`	The direct dependencies with minimum versions, each with a comment on why
`requirements.lock`	Exact versions that passed the tests on your PC (`pip freeze`); the install source for Docker and CI
`requirements-docs.txt`	`python-pptx` , only for building the deck
`.env.example`	Every setting with placeholder values; copy it to `.env` and fill in passwords. `.env` is never committed
`.gitignore`	Keeps `.env` , the venv, generated data and your real data out of Git
`.gitattributes`	Same line endings on Windows and Linux, so files are byte-identical in Docker and CI
`.dockerignore`	Keeps secrets, data, tests and docs out of the Docker image
`Dockerfile`	One image for API, dashboard and tools; Python 3.11 slim, non-root user, telemetry off

File	What it does
`docker-compose.yml`	Starts the database, and with `--profile app` also setup, API and dashboard, all on 127.0.0.1
`.streamlit/config.toml`	Dashboard settings: usage statistics off, listen on 127.0.0.1 only
`.github/workflows/ci.yml`	Three CI jobs: tests against a real PostgreSQL, Docker end-to-end, dependency vulnerability audit

4.3 `src/payops/` (top level)

File	What it does	In more detail
`config.py`	Reads `.env` into validated settings	Refuses any database or Ollama address that isn't local, and Ollama cloud model names; owner password is optional at run time
`netpolicy.py`	The local-only rule	Decides what counts as local (127.x, ::1, localhost, listed container names); installs the network guard that fails any outbound call during tests
`logging_setup.py`	JSON log lines	One line per event, never containing passwords, questions or SQL
`health.py`	`/health` checks	Checks database (read-only login), Ollama and model, and history; returns ok, degraded or down

4.4 `data/` and `db/`

File	What it does	In more detail
`data/scenario.py`	The scenario settings	Providers with their normal failure rates, payment methods, reasons, and the two injected incidents

File	What it does	In more detail
`data/generator.py`	Makes the synthetic dataset	Hour by hour, with evening peaks and growth; writes 4 CSVs, `manifest.json` (fingerprints) and `ground_truth.json` (answer key, never loaded)
`db/schema.sql`	The `payments` tables	providers, provider_incidents, deployments, transactions, daily_stats, dataset_runs; with keys and checks (e.g. a failed payment must have a reason)
`db/loader.py`	Loads a dataset	Checks fingerprints, rebuilds the schema, bulk-loads with `COPY`, builds daily totals, creates the read-only login; all in one transaction
`db/connection.py`	Database connections	Separate engines for the read-only and application logins, with pools and timeouts
`db/history_schema.sql`	`ops` tables for history	investigations and investigation_steps (the checkpoints)
`db/security_schema.sql`	`ops` tables for security	users, sessions, audit_log, plus triggers that block audit edits
`db/setup_app.py`	One-time setup	Creates schema `ops` and the `payops_app` login with exact permissions
`db/setup_history.py`	Older entry point	Forwards to `setup_app` so old commands still work

4.5 `analytics/`

File	What it does	In more detail
`models.py`	Shared shapes	Filters, codes, and `Figure`, which labels each number measured, estimate or hypothetical

File	What it does	In more detail
`queries.py`	All analytics SQL	Parameterised queries run under the read-only login
`metrics.py`	Rates and comparisons	Failure rate, week-vs-week change, which provider explains the extra failures, two-proportion z-test
`detection.py`	Finds anomaly windows	Per provider and hour vs the previous 7 days; flags, merges and filters windows
`detection_eval.py`	Measures detection	Precision and recall across 8 seeds against the answer key
`events.py`	What happened nearby	Deployments and provider notices around a window, and failure rates before vs after each deployment
`impact.py`	Business impact	Value of failed payments (measured), excess value (estimate), handling cost (hypothetical scenarios)
`forecast.py`	Next week's normal	Linear model with weekday and trend, checked against a naive baseline
`report.py`	Weekly report without agents	`python -m payops.analytics.report`; also the template report's basis
`serialize.py`	JSON-safe output	Turns tables into plain values for the API

4.6 `agents/`, `llm/`, `history/`

File	What it does	In more detail
`agents/schemas.py`	The agents' data shapes	Findings, evidence, hypotheses, plan, trace, verification report
`agents/state.py`	Shared workflow state	Lists that parallel steps append to safely; typed for checkpoint validation

File	What it does	In more detail
`agents/tools.py`	Permission-checked tools	Each agent may call only its listed tools; every call recorded; one retry on a dropped connection; 30 calls per step max
`agents/coordinator.py`	Operations Coordinator	Plans (LLM proposal validated by code), assembles findings, asks the LLM to write, chooses LLM vs template report
`agents/transaction_agent.py`	Transaction Intelligence	This week vs last: rates, providers, methods, reasons
`agents/pattern_agent.py`	Pattern Investigation	Anomaly windows, nearby events, hypotheses scored by evidence
`agents/forecast_agent.py`	Forecasting	Next week's volume and normal failure rate
`agents/impact_agent.py`	Business Impact	Labelled money figures for each anomaly
`agents/verification_agent.py`	Verification and Policy	Recomputes figures from the database; checks every LLM sentence for citations, numbers, dates, causal and proof wording
`agents/template_report.py`	Report without the LLM	Clear text report and trace table; the safe fallback
`agents/graph.py`	The LangGraph workflow	9 steps, checkpoint after each, time budget, resume rules
`agents/run.py`	Command line	Ask a question, `--list`, `--show`, `--resume`, `--no-llm`
`llm/client.py`	Talks to Ollama	127.0.0.1 only, JSON-schema replies, temperature 0, timeout, call budget, one retry for invalid output

File	What it does	In more detail
`llm/prompts.py`	The prompts	Question and data inside tags, so text in them is treated as data; rules for citations and wording
`history/store.py`	Saving runs	Postgres and in-memory stores; who created each run; which steps can be reused
`history/codec.py`	Checkpoint format	Encodes state as JSON and validates it on the way back

4.7 `security/`, `api/`, `dashboard/`, `evaluation/`

File	What it does	In more detail
`security/passwords.py`	Password hashing	scrypt, N=2^17, r=8, p=1; constant-time compare; upgrades older hashes
`security/rbac.py`	Roles and permissions	viewer, analyst, lead, admin; admin cannot run investigations
`security/service.py`	Sign-in logic	Sessions (30 min idle, 8 h max), lockout after 5 failures for 15 min
`security/store.py`	Accounts and sessions storage	Postgres and in-memory versions; only a SHA-256 of each token is stored
`security/audit.py`	Audit log	Append-only, each row hashed with the previous one; `verify` finds an altered row
`security/manage.py`	Account commands	create-user, list-users, set-role, disable, reset-password, unlock, audit, verify-audit

File	What it does	In more detail
`api/main.py`	The FastAPI app	Routers, 64 KiB body limit, security headers, error handling without internals
`api/auth.py`	Access checks	Current user from the token, role checks, audit helper
`api/deps.py`	Request helpers	Read-only connection per request; validated filters
`api/routes_auth.py`	Sign-in and admin endpoints	login, logout, me, password, users, audit
`api/routes_analytics.py`	Dashboard data	KPIs, time series, breakdowns, reasons, events, anomalies
`api/routes_investigations.py`	Investigations	Run (max 2 at once), list, open, resume; someone else's run answers 404
`api/schemas.py`	API request and response shapes	Validated by FastAPI
`dashboard/app.py`	The Streamlit page	Sign-in, filters, tabs that follow your role: Trends, Providers, Failure reasons, Incidents & timeline, Investigate, History, Audit log
`dashboard/api_client.py`	Dashboard to API	Local addresses only; one client per signed-in session
`dashboard/charts.py`	Plotly charts	Fixed provider colours; hides noisy points under 30 payments
`evaluation/llm_eval.py`	LLM evaluation	10 fixed questions (ordinary, injection, out of range); plan accuracy, sentences kept, speed

4.8 `tests/`, `scripts/`, `docs/`, `sql/`

File	What it does
`tests/conftest.py`	Shared test setup; blocks internet for every test
`tests/fakes.py`, `fake_llm.py`, `auth_helpers.py`	Helpers: canned query results, a scripted LLM, signing in as a role
`tests/test_*.py`	One file per module, named after it (e.g. `test_verification.py`)
`tests/integration/`	Tests against the real database, loaded data, API, auth, history and live LLM; skipped (not failed) when those aren't running
`scripts/build_presentation.py`	Builds the deck and `docs/presentation_notes.md`
`scripts/prepare_real_data.py`	Profiles, cleans and converts your own data for the loader (section 8)
`scripts/real_data_config.example.json`	Template for mapping your columns and codes
`scripts/prepare_requirements.py`	Reads `requirements.lock` in any Windows encoding for Docker and CI
`scripts/check_test_report.py`	CI gate: fails if tests failed, were skipped unexpectedly, or too few ran
`sql/validate_incident.sql`	Seven plain SQL queries that find the incident by hand
`docs/*.md`	`business_problem`, `architecture` (+ `.svg`), `agent_design`, `analytics_methods`, `data_dictionary`, `security`, `evaluation`, `runbook`, `offline_checklist`, `dependency_inventory`, `limitations_and_future_work`, `demo_script`, `presentation_notes`, `file_guide`

5. What it would change in a finance or payments organisation

The biggest change is speed with evidence: a first-pass investigation that an analyst assembles by hand in hours is produced in under a minute, with every number traceable to a query. The analyst's time moves from gathering data to judging it. These are expected effects; none has been measured in a real company.

Area	Today, by hand	With this system	What it is worth
Time to first answer	Hours of SQL, spreadsheets and status-page checks	About 20–60 s on a laptop	Problems at a provider are escalated sooner, so fewer payments fail
Blaming the wrong cause	The last release is often blamed and rolled back	Hypotheses ranked by evidence; correlation shown as correlation	Fewer needless rollbacks of good work
Talking to providers	"Your failures went up"	Exact window, reason codes, counts and sample transaction IDs	Faster answers from the provider, stronger SLA claims and credits
Money at risk	Rough guesses	Measured, estimated and hypothetical figures kept apart	Finance gets numbers it can rely on, labelled honestly
Audit and compliance	Analysis in personal notebooks	Every run saved with its evidence; every action in a tamper-evident audit log; roles enforced	Easier internal audit and regulator questions
Data protection	Data copied into cloud tools	Nothing leaves the computer; local LLM; pseudonymised customer IDs	Fits strict data-residency rules
Consistency	Each analyst does it differently	Same method every time, tested	Comparable reports week to week

Who would use it: payment operations analysts (run investigations), their leads (review everyone's), support and finance managers (dashboards), and IT or risk (accounts and audit log).

What it does not replace: the analyst's judgment, the provider's own incident report, and the company's incident process. It never acts on its own, for example it doesn't switch traffic between providers.

What a real deployment would still need: a data-protection review before using real customer data; a connection to the company's identity provider with multi-factor sign-in; TLS if it ever leaves one computer; secrets in a vault; audit events copied to write-once storage; per-provider data access for analysts; and a trial on past real incidents to measure accuracy before anyone relies on it. `docs/limitations_and_future_work.md` lists these in order.

6. How to run it: the commands, and why each one

Run everything in PowerShell from the project folder. Part A is done once; on later days start at part B.

```
cd "C:\Users\Jaichiddharth R K\payment-ops-intelligence"
```

A. One-time setup

#	Command	Why
1	`[Environment]::SetEnvironmentVariable("OLLAMA_NO_CLOUD", "1", "User")` then quit and restart Ollama	Switches off Ollama's cloud models, so the LLM can only run locally
2	`ollama pull qwen3:4b`	Downloads the model once (about 2.5 GB). After this, no internet is needed
3	`py -3.11 -m venv .venv`	Creates a private Python environment for this project, so its packages don't clash with others
4	`.\.venv\Scripts\Activate.ps1`	Switches this window to that environment. Repeat in every new window. If refused: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once

#	Command	Why
5	`pip install -r requirements.lock`	Installs the exact package versions that passed the tests on your PC
6	`pip install -e . --no-deps`	Registers the project's own code, so `python -m payops...` commands work
7	`pip install -r requirements-docs.txt`	Only for building the presentation
8	`Copy-Item .env.example .env`	Creates your private settings file (never committed)
9	`python -c "import secrets; print(secrets.token_urlsafe(24))"` three times	Makes three random passwords; paste them into `.env` as `POSTGRES_PASSWORD`, `READONLY_DB_PASSWORD`, `APP_DB_PASSWORD`
10	`docker compose up -d db`	Starts PostgreSQL in Docker, reachable only from this computer
11	`python -m payops.data.generator --seed 42 --out data/generated`	Creates the synthetic dataset with the hidden incident (10–30 s)
12	`python -m payops.db.loader --data-dir data/generated`	Checks the files' fingerprints, loads them, creates the read-only login
13	`python -m payops.db.setup_app`	Creates the history, accounts and audit tables and the application login
14	`python -m payops.security.manage create-user admin --role admin`	First admin account (asks for a password of 12+ characters)
15	`python -m payops.security.manage create-user ana --role analyst`	An analyst account, the one you demo with

B. Every day: start it

Window	Command	Why
1	`docker compose up -d db`	Starts the database (data is kept between days)
1	`ollama ps`	Checks Ollama is running; it starts with Windows
2	`uvicorn payops.api.main:app --host 127.0.0.1 --port 8000`	Starts the API; leave this window open
3	`streamlit run src/payops/dashboard/app.py`	Starts the dashboard; open http://127.0.0.1:8501 and sign in
any	`Invoke-RestMethod http://127.0.0.1:8000/health | ConvertTo-Json -Depth 4`	Confirms database, LLM and history are all `ok`

Activate the venv (`.\.venv\Scripts\Activate.ps1`) in windows 2 and 3 first.

C. Without the dashboard (command line)

Command	Why
`python -m payops.analytics.report`	The weekly report from statistics only, no LLM
`python -m payops.agents.run "What went wrong with Kestrel in the week of 21 September 2026?"`	A full investigation with all six agents and the LLM
add `--no-llm`	Same, with the template report, in under a second
`python -m payops.agents.run --list`	Saved investigations
`python -m payops.agents.run --resume <id>`	Continue an interrupted run; replace `<id>` with a real ID from `--list`
`python -m payops.evaluation.llm_eval`	Measures the LLM on 10 questions (3–5 minutes)

Command	Why
`python -m payops.security.manage verify-audit`	Checks nobody altered the audit log
`python scripts/build_presentation.py`	Rebuilds the deck

D. Check everything works

Command	Why
`pytest`	Runs all tests. Expected: **448 passed** with the database, Ollama and python-pptx available. Run it on the synthetic seed-42 data: the live tests expect the hidden KESTREL incident
`Get-Content sql\validate_incident.sql | docker exec -i payops-db psql -U payops_admin -d payops`	Finds the incident with plain SQL, without the app

E. The packaged way (everything in Docker)

Command	Why
Stop uvicorn and Streamlit (Ctrl+C) first	They use the same ports, 8000 and 8501
`docker compose --profile app up -d --build`	Builds the image and starts setup, API and dashboard
`docker compose --profile app ps`	db, api and dashboard healthy; setup "exited (0)" is normal
`docker compose --profile app down`	Stops the containers and keeps the data

In Docker the reports use the labelled template, because containers can't reach Ollama on 127.0.0.1. For LLM demos, use part B.

F. Stop for the day

Ctrl+C in the API and dashboard windows, then `docker compose stop db`, then `deactivate`. Your data and history stay.

7. How to demonstrate it

A strong demo takes about 20 minutes: 12–15 minutes of slides (`presentation/payment_ops_intelligence.pptx`, speaker notes included) and a 6–8 minute live demo with Wi-Fi switched off. The full script with fallbacks is `docs/demo_script.md`.

Before the audience arrives (10 minutes)

1. Start part B of section 6 (database, API, dashboard).
2. Warm the model so the first answer is fast: `ollama run qwen3:4b "hi"`.
3. Check `/health` shows everything `ok`.
4. Sign in as `ana` and run one investigation, so the History tab has something to show.
5. Switch Wi-Fi off and reload the dashboard. It still works: that is your first proof point.

The live demo

Time	Do	Say
0:00	Sign in as `ana`	"Local accounts, checked by the API. Wi-Fi is off; everything runs on this laptop."
0:30	Trends and Providers tabs	"The failure rate jumps around 23 September; Kestrel, in orange, stands out. No AI yet, just SQL."
1:30	Investigate tab: *What went wrong with Kestrel in the week of 21 September 2026?*	"Six agents: the LLM plans, three agents analyse in parallel, impact is sized, the LLM writes, every sentence is checked."
2:30	Headline and **Hypotheses**	"Kestrel went from about 3% to 23% failures in the window. Our own release two hours earlier is ranked unlikely: only Kestrel got worse."

Time	Do	Say
3:30	**Verification** and **Findings and evidence**	"Every sentence cites its findings; numbers and dates are checked; key figures are recomputed."
4:30	History tab, *Resume from checkpoints*	"Each step is saved; an interrupted run continues where it stopped."
5:15	Sign out, sign in as `admin`	"No Investigate tab: admins manage accounts but can't run investigations. The API refuses it too."
5:45	Audit log, **Verify the hash chain**	"Every action is recorded; a changed row would be found."

If something goes wrong

- **The LLM is slow or off:** the report says "template report"; explain that this is the designed fallback. Or run `python -m payops.agents.run "Why did the failure rate increase last week?" --no-llm`.
- **The dashboard won't load:** use the command line from section 6C.
- **Port already in use:** the Docker containers are running: `docker compose --profile app down`.
- **Account locked:** `python -m payops.security.manage unlock ana`.

Questions you will likely get

Question	Short answer
Why not GPT-4 or Claude through an API?	Project rule: no paid APIs and no data leaving the computer. The design makes a small local model safe enough
How do you know it's right?	The synthetic data has an answer key the app never reads; tests and the evaluation compare against it
What stops the LLM inventing numbers?	It never computes them; any number it writes must appear in the finding it cites, or the sentence is removed

Question	Short answer
Can it be wrong anyway?	Yes: a right number can be given the wrong meaning. That's why the evidence sits next to every sentence
Does it work on real data?	The pipeline does (section 8); its accuracy on real incidents has not been measured yet

Demonstrating on your own data

Prepare and load it as in section 8, then ask about a week you know had a problem. Two things change: the question names your providers and dates, and you can't show an answer key, so compare the report with what your team already knows happened. Reload the synthetic data afterwards before running `pytest`.

8. Using real data

You do not change the application code to use real data. Put your exports in `data/raw/`, describe your columns in one config file, and run `scripts/prepare_real_data.py`. It profiles the data (EDA), cleans it, pseudonymises customer IDs and writes the exact files the loader needs. The agents read the provider list and dates from the database, so they adapt automatically.

8.1 The steps

1. **Export** at least 35 days of payment attempts (90 is better) as CSV, plus your release history and provider status notices if you have them. Remove card numbers, names, emails and phone numbers at the source; this project never needs them.
2. **Place** the files in `data/raw/` (create it). This folder is git-ignored.
3. **Copy the config:** `Copy-Item scripts\real_data_config.example.json data\raw\real_data_config.json`, then edit it (8.3).
4. **Set a secret salt** for pseudonymising customer IDs: `$env:REAL_DATA_SALT = python -c "import secrets; print(secrets.token_urlsafe(32))"`. Save that value in a password manager: the same salt gives the same pseudonyms next month.
5. **Prepare:** `python scripts/prepare_real_data.py --config data/raw/real_data_config.json --out data/real`

6. **Read** `data/real/eda_report.md` and `data/real/rejected_rows.csv`. Fix the config and repeat step 5 until the warnings make sense (8.4).
7. **Load:** `python -m payops.db.loader --data-dir data/real`. This replaces the synthetic data; history, accounts and the audit log are kept.
8. **Validate** (8.6), then use the dashboard and agents as usual.
9. **Switch back** for tests or the standard demo: run the generator and loader for seed 42 again (section 6A, steps 11–12).

8.2 What your transactions file must contain

One row per payment **attempt** (a retry is a new row). Your column names can be anything; the config maps them.

Field the project needs	Required	What it is	Cleaning applied
`transaction_id`	yes	Unique ID per attempt	Duplicates removed (first kept); optional hashing
`created_at`	yes	When the attempt was made	Parsed in `source_timezone`, converted to UTC; unreadable rows removed
`provider_code`	yes	Gateway or acquirer that processed it	Upper case, spaces to `_` ("Gateway A" → `GATEWAY_A`)
`payment_method`	yes	Card, UPI, net banking…	Mapped to `CARD`, `BANK_TRANSFER`, `WALLET`, `NET_BANKING`
`amount`	yes	Payment amount	Commas removed; zero or negative removed; converted to one currency
`status`	yes	Final outcome	Mapped to `SUCCEEDED` or `FAILED`; pending, refunds, unknown removed
`failure_reason`	for failures	Decline or error code	Mapped to the 7 reason codes; unmapped → `PROVIDER_ERROR` (counted)

Field the project needs	Required	What it is	Cleaning applied
`currency`	recommended	ISO code	Converted with `fx_rates`; currencies without a rate removed
`customer_id`	optional	Customer reference	Replaced by `CUST-` + HMAC pseudonym
`latency_ms`	optional	Gateway response time	Negative removed; 0 if you don't have it
`provider_event_id`	optional	Provider incident ID in the response	Kept only if that incident is in the incidents file

Optional files: deployments (`deployment_id` , `service` , `deployed_at` , plus `version` , `environment` , `team` , `change_summary`) and provider incidents (`incident_id` , `provider_code` , `reported_start_at` , `title` , plus `reported_end_at` , `severity` , `description`). Without deployments the Pattern agent can't test your releases as a cause; without incidents reports can't cite provider notices.

8.3 Editing the config: what to replace

Setting	Replace with	Why it matters
`transactions_file` , `deployments_file` , `incidents_file`	Your file paths (or `null`)	
`columns`	Your column name for each field (`null` if missing)	A wrong name stops the script with a list of the real columns
`source_timezone`	The time zone your export uses, e.g. `Asia/Kolkata`	Wrong zone shifts every incident window by hours
`currency` , `fx_rates`	Keep `USD` ; add a rate per currency, e.g. `"INR": 0.0113` (USD per INR, for the period)	Impact figures are labelled USD and the handling-cost scenarios are USD amounts

Setting	Replace with	Why it matters
`amount_in_minor_units`	`true` if amounts are in paise or cents	Otherwise every amount is 100× too big
`status_map`	Every final status your system uses	The most important mapping: a wrong one changes every failure rate
`method_map`	Your method names	Unmapped methods are removed, not guessed
`reason_map`	Your decline and error codes, e.g. ISO 8583 `51` → `INSUFFICIENT_FUNDS`, `91` → `TIMEOUT`	Separates customer declines from technical failures; the agents treat `TIMEOUT` and `PROVIDER_ERROR` as provider-side evidence
`provider_names`	Display names, e.g. `"GATEWAY_A": "Gateway A"`	Shown in the dashboard
`start`, `end`	Optional date limits	Cut off a partial first or last day

8.4 EDA and cleaning: what to look at before loading

The script writes four sections to `eda_report.md`: the raw profile, what cleaning changed, the cleaned profile and warnings. Check these, in order:

- [ ] **Raw status values:** every value is either in `status_map` or meant to be excluded (pending, refunded, voided)
- [ ] **Rows kept:** above 90%. If lower, read `rejected_rows.csv` grouped by `_rejected_because`
- [ ] **Unmapped failure reasons** (the "set to PROVIDER_ERROR" row): small, or extend `reason_map`. A catch-all hides causes
- [ ] **Days covered:** 35 or more, no empty days, no days under 30% of normal volume (partial exports look like outages)
- [ ] **Per-provider failure rates:** plausible (usually a few percent). Above 50% usually means pending or abandoned attempts counted as failures

- [ ] **Low-volume providers:** under 50 payments a day give noisy hourly rates; small incidents there won't be detected
- [ ] **Amounts:** maximum not wildly above the 99th percentile (test payments, or minor and major units mixed)
- [ ] **Busiest hour (UTC):** matches your real peak once converted. If not, `source_timezone` is wrong
- [ ] **Retries and duplicates:** if your system logs one payment several times (webhook replays), deduplicate on the right ID
- [ ] **Test traffic:** remove sandbox merchants and internal test cards before export

8.5 Code you may change (optional)

File	Change	When
`src/payops/dashboard/charts.py`	`PROVIDER_COLORS`: your provider codes	Otherwise every provider is grey
`src/payops/analytics/impact.py`	`ImpactAssumptions`: cost per failure scenarios	When finance gives you real handling costs
`src/payops/analytics/detection.py`	Thresholds ($z \geq 4$, 5 extra failures, 15 per window)	Only after checking against known past incidents
`src/payops/evaluation/llm_eval.py`	The 10 questions	They are written for the synthetic data; write your own with known answers

Leave `data/scenario.py` and `data/generator.py` alone: they only make the synthetic data.

8.6 Validate after loading

- `python -m payops.analytics.report`: does the week's summary match what your team saw?

- Pick a past incident you know well and ask the agents about that week. Did they find the right provider, reason and window? This is your accuracy check, since real data has no answer key.
- Open the dashboard: provider list, dates and volumes look right.
- Expect `pytest` live tests to fail on real data: they check for the synthetic KESTREL incident. Reload seed 42 before testing.

8.7 Privacy rules

- Never put card numbers, CVV, names, emails, phone numbers or addresses in the export.
- `data/raw/` and `data/real/` are git-ignored. Check `git status` before every commit anyway.
- `rejected_rows.csv` contains raw rows: delete it once you've read it.
- Get your organisation's data-protection approval before using real customer data, even on one laptop.